
嵌入式系统实验指导书

试验 5

2025 年 9 月

目录

1. 实验五：使用 RUST 进行嵌入式开发实验.....	3
1.1 RUST 嵌入式开发简介.....	3
1.1.1 RUST 用于嵌入式开发的特点.....	4
1.1.2 核心抽象：PAC、HAL 与 EMBEDDED-HAL.....	6
1.1.3 NO_STD 环境与生态.....	7
1.2实验一 LED实验.....	8
1.2.1 实验目的	8
1.2.2 实验环境	8
1.2.3 实验原理	9
1.2.4 实验内容	10
1.2.5 实验步骤	13
1.2.7 实验现象	20

1. 实验五：使用 Rust 进行嵌入式开发实验

1.1 Rust 嵌入式开发简介

Rust 是一种现代化的、注重安全、并发和性能的系统编程语言。近年来，它在嵌入式领域的应用日益广泛，被认为是传统 C/C++ 语言的有效替代者。Rust 嵌入式开发旨在将 Rust 语言的内存安全、零成本抽象和富有表现力的类型系统带入资源受限的微控制器（MCU）世界，从而显著提升嵌入式软件的可靠性和开发效率。

与依赖于完整操作系统的开发不同，Rust 嵌入式开发通常从“裸机”（Bare-metal）层面开始，即不依赖任何预设的底层软件。开发者通过一系列精心设计的库（在 Rust 中称为 "crate"）来直接与硬件交互。这种方法为开发者提供了极高的灵活性和对系统资源的终极控制能力。

Rust 适用于从简单的 8 位 MCU 到功能强大的 64 位多核处理器的各种嵌入式系统。社区已经为众多主流硬件平台（如 STM32、nRF、ESP32、Raspberry Pi Pico 等）提供了成熟的开发支持。

1.1.1 Rust 用于嵌入式开发的特点

1. 内存安全 (Memory Safety)

Rust 的核心特性是其所有权（Ownership）、借用（Borrowing）和生命周期（Lifetimes）系统。这套机制在编译阶段就能静态地保证内存安全，彻底消除了空指针解引用、数据竞争、缓冲区溢出等在 C/C++ 中常见的运行时错误。对于要求高可靠性的嵌入式系统而言，这是一个革命性的进步。

2. 零成本抽象 (Zero-cost Abstractions)

Rust 允许开发者编写高级、易读的代码，而无需担心性能损失。它的泛型、Trait（类似于接口）和宏等抽象机制在编译时会被优化掉，最终生成的机器码与手写的低级 C 代码一样高效。这意味着你可以使用高级的编程范式来操作硬件，同时保持最佳性能。

3. 无畏并发 (Fearless Concurrency)

Rust 的类型系统能够理解数据在不同线程间的传递，并强制执行线程安全协议。这使得编写并发代码（尤其是中断服务程序）变得更加安全。编译器会检查并阻止潜在的数据竞争问题，让你自信地编写和重构访问共享硬件资源的代码。

4. 现代化的工具链和包管理

Rust 提供了统一且强大的工具链。rustup 用于管理 Rust 版本和交叉编译工具链，Cargo 作为构建系统和包管理器，极大地简化了项目依赖管理、编译、测试和文档生成过程。crates.io 是 Rust 社区的中央包仓库，提供了大量可重用的嵌入式库。

5. 优秀的 C 语言互操作性 (FFI)

Rust 提供了优秀且无缝的外部函数接口（FFI），可以轻松地调用现有的 C 语言库（如厂商提供的驱动库、协议栈等），也能够将 Rust 代码编译成 C 兼容的库。这使得将 Rust 渐进式地引入现有项目成为可能。

6. 活跃的社区生态

Rust 嵌入式工作组（Embedded WG）和活跃的社区为开发者提供了丰富的资源，包括项目模板、驱动程序、协议实现和详尽的文档，显著降低了入门门槛。

1.1.2 核心抽象：PAC、HAL 与 embedded-hal

为了在不同厂商的微控制器之间实现代码的可移植性和重用性，Rust 嵌入式生态系统建立了一套标准化的分层抽象模型。

1. 外设访问层 (Peripheral Access Crate - PAC)

PAC 是最底层的硬件抽象，它提供了对微控制器所有寄存器的类型安全访问。通常，PAC 是使用 `svd2rust` 等工具根据芯片厂商提供的 SVD (System View Description) 文件自动生成的。它将每个寄存器、每个字段都映射为 Rust 的结构体和方法，从而防止了因直接写入错误内存地址或非法数值而导致的硬件故障。

优点：绝对的底层控制，类型安全，零开销。

缺点：API 过于繁琐，与特定芯片型号高度绑定，不具备可移植性。

2. 硬件抽象层 (Hardware Abstraction Layer - HAL)

HAL 构建在 PAC 之上，为外设（如 GPIO、USART、I2C、SPI 等）提供了更高级、更易于使用的 API。它封装了繁琐的寄存器配置细节（如时钟使能、引脚复用等），向开发者呈现一个面向任务的接口。本实验使用的 `stm32f1xx-hal` 就是一个 HAL 库。

优点：API 设计人性化，简化了开发流程。

缺点：仍然与特定的芯片家族（如 `stm32f1xx-hal` 对应 STM32F1 系列）绑定。

3. embedded-hal Trait

`embedded-hal` 是整个生态系统的基石。它定义了一系列标准的 Trait（接口），用于描述通用外设的行为。例如，它定义了 `OutputPin Trait`，包含 `set_high()` 和 `set_low()` 等方法。

各个芯片的 HAL crate 会为自己的外设实现这些标准 Trait。这使得上层的驱动程序（如传感器驱动、显示屏驱动）可以完全基于 `embedded-hal Trait` 进行开发，而无需关心底层是何种型号的微控制器。只要该 MCU 的 HAL 实现了标准 Trait，驱动程序就可以“开箱即用”。

1.1.3 no_std 环境与生态

嵌入式系统通常没有操作系统的支持，因此也无法使用依赖于操作系统的标准库（std），例如文件 I/O、网络、线程、堆内存分配器等。

Rust 通过 `#![no_std]` 属性来支持在这种环境下编程。在 `no_std` 模式下：

- **std 库不可用**：所有依赖操作系统的功能都无法使用。
- **core 库可用**：`core` 是 `std` 的一个子集，提供了不依赖于操作系统的核心功能，如基本数据类型（`u32`, `bool`）、`Option` 和 `Result`、迭代器、闭包以及原子操作等。
- **alloc 库可选**：如果需要动态内存分配（如 `Box`, `Vec`），可以引入 `alloc` 库并提供一个全局内存分配器的实现。
- **程序入口点**：在 `no_std` 环境下，没有传统的 `main` 函数。开发者需要通过 `cortex-m-rt` 等运行时 `crate` 定义程序的入口点（`#[entry]`）和异常处理函数。

这个看似受限的环境实际上催生了一个强大的 `no_std` 生态系统。社区开发了大量可在 `no_std` 环境下运行的 `crate`，涵盖了数据结构、算法、序列化、加密等领域，为裸机开发提供了强大的支持。

1.2 实验一 LED 实验

1.2.1 实验目的

1. 学习如何搭建 Rust 嵌入式交叉编译开发环境。
2. 了解一个基本的 Rust 嵌入式项目的结构。
3. 学会在 STM32F103CB 平台上编译、烧录并运行一个简单的“点灯”程序。

1.2.2 实验环境

1. 硬件：硬件：STM32 无线节点板一个、下载调试板、ARM JLink 仿真器、USB 方口线、PC 机、MicroUSB 线/交叉串口线、DC5V 电源；
 2. 软件：Windows7 及以上系统
- Rust 工具链（通过 rustup 安装）。
 - 交叉编译目标：thumbv7m-none-eabi。
 - 烧录与调试工具：J-LINK。
 - 代码编辑器（推荐 VS Code + rust-analyzer 插件）。

1.2.3 实验原理

本实验的目标是在 STM32 开发板上实现板载 LED 的周期性闪烁。其基本原理如下：

1. 交叉编译：我们在 PC（通常是 x86-64 架构）上编写 Rust 代码，然后使用 Rust 编译器将其交叉编译为能在目标微控制器（ARM Cortex-M3 架构）上运行的机器码。编译的最终产物是一个 ELF 格式的可执行文件。
2. 硬件初始化：程序开始执行时，首先需要对微控制器进行最基本的初始化。这包括设置系统时钟（RCC - Reset and Clock Control），为外设提供工作的时钟源。
3. GPIO 配置：板载 LED 通常连接到微控制器的一个通用输入/输出（GPIO）引脚上（在 Blue Pill 上是 PA0）。我们需要将这个引脚配置为推挽输出模式（Push-pull output），这样程序才能通过控制该引脚的电平来点亮或熄灭 LED。
4. 主循环：在完成初始化后，程序进入一个无限循环（loop）。在循环体内，我们交替执行以下操作：
 - 将 PA0 引脚设置为特定电平，点亮 LED。
 - 进行一段延时。
 - 将 PA0 引脚设置为相反电平，熄灭 LED。
 - 再进行一段延时。
5. 烧录与运行：编译好的 ELF 文件通过 ST-Link 烧录器和 probe-rs 工具写入到 STM32 的 Flash 存储器中。烧录完成后，probe-rs 会复位芯片，程序开始从入口点运行，我们即可观察到 LED 闪烁的现象。

1.2.4 实验内容

1. 安装 Rust 工具链和必要组件

- 1) 安装 rust（需要安装过 VS Studio 的 C++ 生成工具），打开 rustup-init.exe，进入安装终端后按 Enter 执行默认安装，如图 1-1，安装过程会自动执行，直到安装完成，如图 1-2。

```
The Cargo home directory is located at:
C:\Users\...\scoop\persist\rustup\.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:
C:\Users\...\scoop\persist\rustup\.cargo\bin

This path will then be added to your PATH environment variable by
modifying the PATH registry key at HKEY_CURRENT_USER\Environment.

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:

  default host triple: x86_64-pc-windows-msvc
  default toolchain:   stable (default)
  profile:             default
  modify PATH variable: yes

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
```

图 1-1 安装 rust

```
Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload its PATH environment variable to include
Cargo's bin directory (C:\Users\...\scoop\persist\rustup\.cargo\bin).

Press the Enter key to continue.
```

图 1-2 安装 rust 完成

- 2) 检查是否正确安装了 Rust，可打开终端并输入下面两行，此时能看到最新发布的稳定版本的版本号、提交哈希值和提交日期，如图 1-3

```
rustc -V
cargo -V
```

```
(base) PS C:\Windows\System32> rustc -V
rustc 1.90.0 (1159e78c4 2025-09-14)
(base) PS C:\Windows\System32> cargo -V
cargo 1.90.0 (840b83a10 2025-07-30)
(base) PS C:\Windows\System32> |
```

图 1-3 检查是否正确安装了 Rust

- 3) 创建 Rsut 项目，自行打开终端选择一个目录作为起点（不要选择打开终端时的默认目录），后者直接在一个自选文件夹中右键选择“在终端中打开”，如图 1-3，输入以下命令创建名为” cargo new stm32f103cb-blink”的 Rsut 项目，并在终端中进入新建好的项目文件夹（如图 1-4）：

```
cargo new stm32f103cb-blink
cd stm32f103cb-blink
```

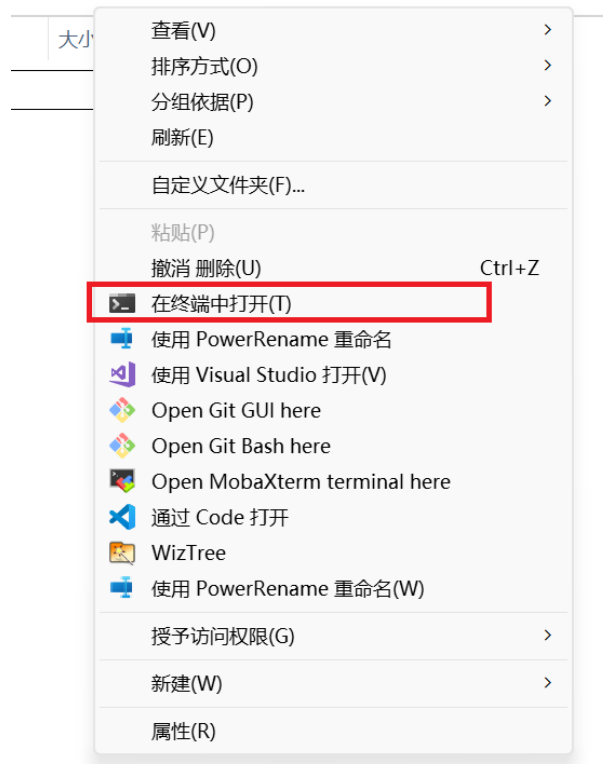


图 1-4 打开终端

```
(base) PS D:\MyProjects\STM32projects\25course> cargo new stm32f103cb-blink
Creating binary (application) 'stm32f103cb-blink' package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
(base) PS D:\MyProjects\STM32projects\25course> cd stm32f103cb-blink
(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> |
```

图 1-5 创建新项目

- 4) 在文件夹中打开新建的项目，项目结构如图 1-6

名称	修改日期	类型	大小
.git	2025/10/2 17:11	文件夹	
src	2025/10/2 17:11	文件夹	
.gitignore	2025/10/2 17:11	Git Ignore 源文件	1 KB
Cargo.toml	2025/10/2 17:11	Toml 源文件	1 KB

图 1-6 项目结构

5) 打开 Cargo.toml，把里面的内容替换为如下内容：

```
[package]
name = "stm32f103cb-blink"
version = "0.1.0"
edition = "2021"

[dependencies]
embedded-hal = "1.0"
nb = "1"
cortex-m = "0.7.7"
cortex-m-rt = "0.7.5"
panic-halt = "1.0.0"

[dependencies.stm32f1xx-hal]
version = "0.11.0"
features = ["stm32f103", "medium"]
```

6) 打开 src/main.rs，把里面的内容替换为如下内容：

```
#![在 PA0 引脚上闪烁一个 LED]

#![deny(unsafe_code)]
#![no_std]
#![no_main]

use panic_halt as _;

use nb::block;

use cortex_m_rt::entry;
use stm32f1xx_hal::{pac, prelude::*, timer::Timer};

#[entry]
fn main() -> ! {
    // 从 cortex-m crate 获取对内核外设的访问权限
    let cp = cortex_m::Peripherals::take().unwrap();
    // 从外设访问 crate (PAC) 获取对芯片特定外设的访问权限
    let dp = pac::Peripherals::take().unwrap();

    let mut rcc = dp.RCC.constrain();
```

```
// 获取 GPIOA 外设
let mut gpioa = dp.GPIOA.split(&mut rcc);

// 将 GPIOA 的第 0 号引脚 (PA0) 配置为推挽输出模式。
// 由于 PA0 属于引脚 0-7 范围, 因此使用 `cr1` 寄存器进行配置。
let mut led = gpioa.pa0.into_push_pull_output(&mut gpioa.cr1);

// 配置系统滴答定时器 (SysTick), 使其每秒产生一次更新事件
let mut timer = Timer::syst(cp.SYST, &rcc.clocks).counter_hz();
timer.start(1.Hz()).unwrap();

// 等待定时器触发更新事件, 并切换 LED 的状态
loop {
    block!(timer.wait()).unwrap();
    led.set_high(); // 点亮 LED
    block!(timer.wait()).unwrap();
    led.set_low(); // 熄灭 LED
}
}
```

7) 新建文件 memory.x, 输入以下内容:

```
/* Linker script */
MEMORY
{
    FLASH : ORIGIN = 0x08000000, LENGTH = 64K
    RAM : ORIGIN = 0x20000000, LENGTH = 20K
}
```

8) 新建文件夹.cargo, 并在里面新建 config.toml, 输入以下内容:

```
[build]
target = "thumbv7m-none-eabi"

[target.thumbv7m-none-eabi]
rustflags = ["-C", "link-arg=-Tlink.x"]
```

1.2.5 实验步骤

1) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机, 用 20 针排线连接 ARM J-LINK 仿真器和下载调试板, 10 针排线连接下载调试板和 STM32 开发板, 如 1-7 所示。



图 1-7 硬件连接图

2) 在终端中输入以下命令安装 arm 目标支持，安装完成如图 1-8

```
rustup target install thumbv7m-none-eabi
```

```
(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> rustup target install thumbv7m-none-eabi
info: downloading component 'rust-std' for 'thumbv7m-none-eabi'
info: installing component 'rust-std' for 'thumbv7m-none-eabi'
```

图 1-8 安装 arm 目标支持

3) 在终端中输入以下命令进行项目编译，编译完成后如图 1-9 所示

```
cargo build
```

```

Compiling portable-atomic v1.11.1
Compiling vcell v0.1.3
Compiling void v1.0.2
Compiling heapless v0.8.0
Compiling stable_deref_trait v1.2.0
Compiling byteorder v1.5.0
Compiling bitfield v0.13.2
Compiling critical-section v1.2.0
Compiling gcd v2.3.0
Compiling stm32f1 v0.16.0
Compiling stm32-usbd v0.8.0
Compiling stm32f1xx-hal v0.11.0
Compiling bitflags v1.3.2
Compiling embedded-hal v1.0.0
Compiling embedded-io v0.6.1
Compiling panic-halt v1.0.0
Compiling nb v0.1.3
Compiling volatile-register v0.2.2
Compiling embedded-can v0.4.1
Compiling embedded-dma v0.2.0
Compiling embedded-hal v0.2.7
Compiling fugit v0.3.7
Compiling semver v0.9.0
Compiling hash32 v0.3.1
Compiling bxcan v0.8.0
Compiling embedded-hal-nb v1.0.0
Compiling rustc_version v0.2.3
Compiling fugit-timer v0.1.3
Compiling bare-metal v0.2.5
Compiling usb-device v0.3.2
Compiling syn v2.0.106
Compiling cortex-m-rt-macros v0.7.5
Compiling stm32f103cb-blink v0.1.0 (D:\MyProjects\STM32projects\25course\stm32f103cb-blink)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 6.71s

```

图 1-9 项目编译成功

4) 在终端中输入以下命令安装 hex 转换包，如图 1-10

```

rustup component add llvm-tools
cargo install cargo-binutils

(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> rustup component add llvm-tools
info: component 'llvm-tools' for target 'x86_64-pc-windows-msvc' is up to date
(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> cargo install cargo-binutils
    Updating crates.io index
    Ignored package 'cargo-binutils v0.4.0' is already installed, use --force to override

```

图 1-10 安装 hex 转换包

5) 在终端中输入以下命令编译项目，并生成用于烧录的 flash.hex，如图 1-11，生成的 hex 文件在项目目录下，如图 1-12

```

cargo build
cargo objcopy --release -- -O ihex flash.hex

```

```
(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> cargo build
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.06s
(base) PS D:\MyProjects\STM32projects\25course\stm32f103cb-blink> cargo objcopy --release -- -O ihex flash.hex
Finished `release` profile [optimized] target(s) in 0.03s
```

图 1-11 编译项目

 .cargo	2025/9/30 22:45	文件夹	
 .git	2025/10/2 17:20	文件夹	
 src	2025/10/2 17:20	文件夹	
 target	2025/10/2 17:45	文件夹	
 .gitignore	2025/10/2 17:20	Git Ignore 源文件	1 KB
 Cargo.lock	2025/10/2 17:26	LOCK 文件	9 KB
 Cargo.toml	2025/10/2 17:22	Toml 源文件	1 KB
 flash.hex	2025/10/2 17:54	HEX 文件	10 KB
 memory.x	2025/10/2 17:32	X 文件	1 KB

图 1-12 项目编译完成后项目生成 hex 文件

6) 在项目目录新建 jlink 配置文件 flash.jlink:

```
; flash_hex.jlink
device STM32F103CB
si 1
speed 4000
r
h
erase
loadfile flash.hex
r
g
exit
```


7) 在项目目录新建安装脚本 flash.bat:

```
@echo off
setlocal enableextensions

for %%K in ("HKLM\SOFTWARE\SEGGER\J-Link" ^
           "HKLM\SOFTWARE\WOW6432Node\SEGGER\J-Link" ^
           "HKCU\SOFTWARE\SEGGER\J-Link") do (
  for %%V in (Path InstallPath) do (
    for /f "tokens=2,*" %%a in ('reg query "%%~K" /v %%V 2^>nul ^| findstr /I "REG_"') do (
      if exist "%%b\JLink.exe" set "JLINK=%%b\JLink.exe"
    )
  )
  if defined JLINK goto :run
)

echo 未找到 JLink.exe
exit /b 1

:run
echo 使用: "%JLINK%"
"%JLINK%" -CommandFile "%~dp0flash.jlink"
exit /b %errorlevel%
```

8) 安装 JLINK, 打开 Jlink 安装程序, 安装步骤如图 1-13 至图 1-17 所示,

注意图 1-14 不要勾选



图 1-13 J-Link 安装 1

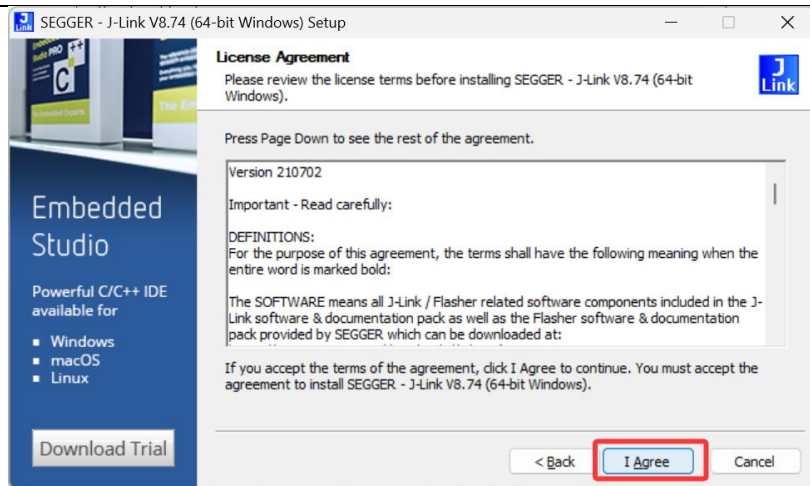


图 1-14 J-Link 安装 2

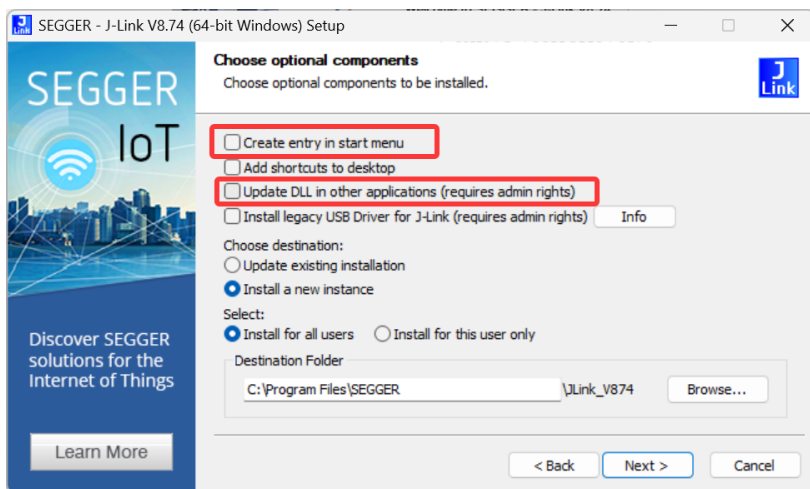


图 1-15 J-Link 安装 3

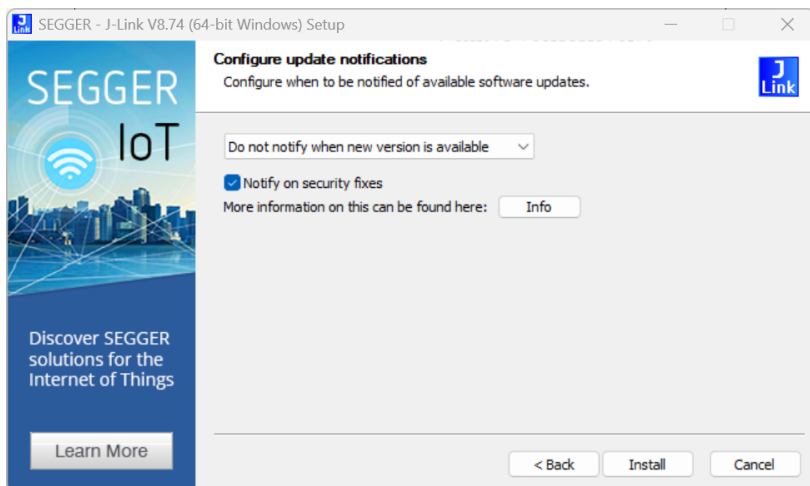


图 1-16 J-Link 安装 4



图 1-17 J-Link 安装完成

9) 回到项目目录，在终端中执行 flash.bat 脚本或者在文件夹中双击 flash.bat 脚本即可进行编译完成后 hex 文件的烧录，如图 1-17

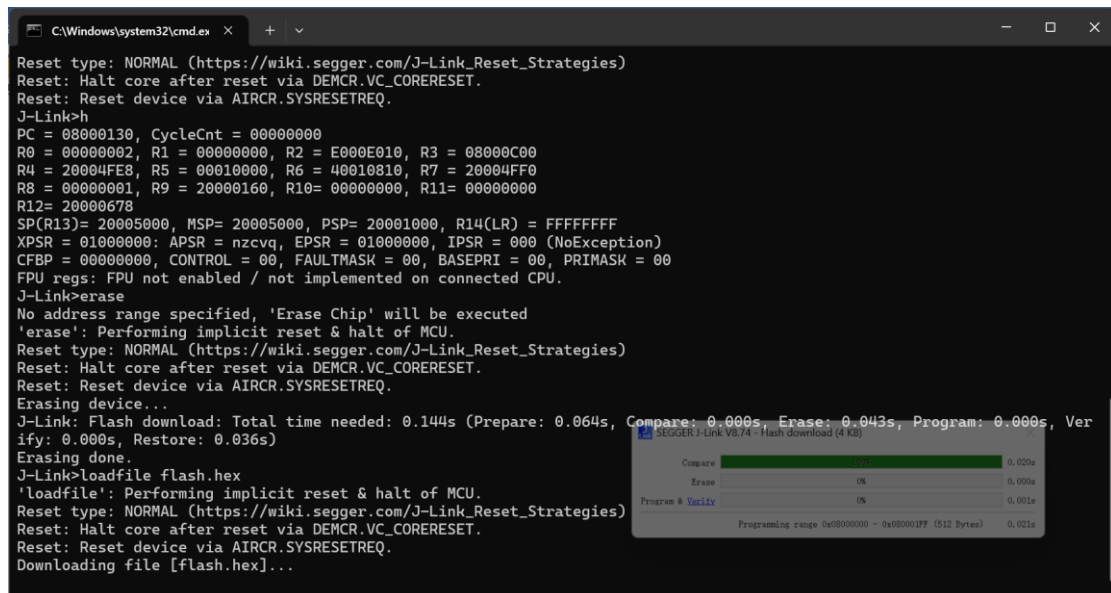


图 1-18 使用脚本进行 J-Link 烧录

10) 观察 STM32 开发板 LED 灯的现象。

1.2.7 实验现象

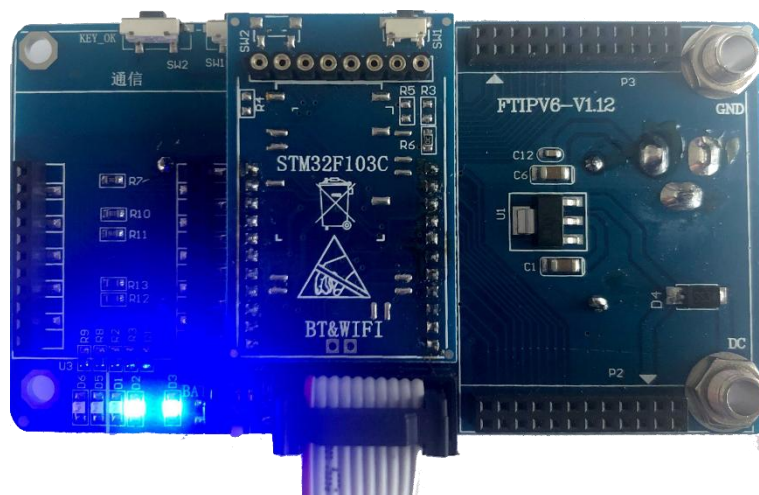


图 1-19 LED 闪烁实验现象

可以看到 STM32 开发板上的 D2 灯闪烁，D2 灯亮灭的时间间隔为 1s，如图 1-19 所示。